

Module Detail		Trainee's Detail		
SECTOR:	ICT	Reg No:	24RP15022	
SUB-SECTOR:	Information Technology	Class:	Level 8 Information Technology	
		Trainer's Detail		
CERTIFICATE:	Bachelors of Technology	Name:	Jean Marie Vianney NYIRIMANA	
MODULE (Code &Title):	ITLDM801 - DATA MINING AND DATA WAREHOUSING	Additional info		
Scored marks:		Decision:	Competent	
			Not Yet Competent	

Store Books ETL and Data Warehouse Project Report

Project Overview:

This project aims to transform the existing Store Books Sales database into a robust data warehouse, enabling advanced analytics and reporting capabilities. The project involves the use of Extract, Transform, Load (ETL) processes to migrate and transform data, and the implementation of SQL Server Analysis Services (SSAS) to create a cube for in-depth data analysis.

Tools and Technologies Used

➤ Database Management Systems (DBMS)

- **Microsoft SQL Server (2022):** for designing and managing the data warehouse.
- **SQL Server Management Studio (SSMS):** for database management and query execution.
- **SQL Queries:** for retrieving data from the **source database** and load them into Datawarehouse.

➤ Data Modeling & Schema Design

- **Star Schema Design:** Used as data warehouse structure.
- **Python:** Data processing and transformation.

How the Store Books Data Warehouse Project implemented

Implement of **Store Books Data Warehouse Project** involves, **ETL (Extract, Transform, Load)** process which is a critical part of creating a data warehouse.

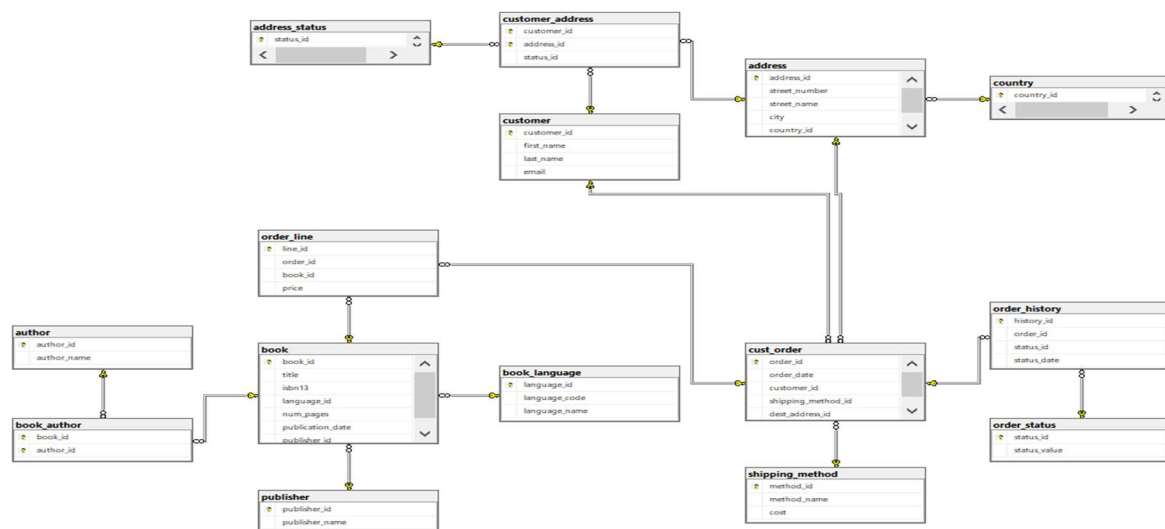
ETL involves:

- Extracting data from the source system
- Transforming data to fit the warehouse schema,
- Loading data into the data warehouse

1. Extracting data from the source system

The Extract phase focuses on retrieving data from the **gravity_books** database. This step ensures that the necessary raw data is gathered for further processing and transformation.

Figure 1: Data source schema



Data from Source Tables Database (gravity_books)

Import necessary libraries

```
import pyodbc
import pandas as pd
from datetime import datetime
import warnings
warnings.filterwarnings("ignore", category=UserWarning, message="pandas only supports
SQLAlchemy connectable")
```

Database Connection

```
conn = pyodbc.connect("
    DRIVER={ODBC Driver 17 for SQL Server};
    SERVER=DESKTOP-MF69IV4\\SQLEXPRESS;
    DATABASE=gravity_books;
    Trusted_Connection=yes
")
cursor = conn.cursor()
```

Retrieve Data from Source Tables

```
cursor.execute("SELECT * FROM table_name")
rows = cursor.fetchall()
data = [list(row) for row in rows]
table_column_names = [column[0] for column in cursor.description]
# Define your custom column names
custom_column_names = ['column_name1', 'column_names2']
# Create a Pandas DataFrame with your custom column names
retrievedData = pd.DataFrame(data, columns=table_column_names)
retrievedData
```

Data from book table [PYTHON CODES]

```
cursor.execute("SELECT * FROM book")
rows = cursor.fetchall()
data = [list(row) for row in rows]
book_column_names = [column[0] for column in cursor.description]
# Define your own column names
custom_column_names = ['book_id', 'title', 'isbn13', 'language_id', 'num_pages', 'publication_date',
'publisher_id']
# Create a Pandas DataFrame with your custom column names
bookData = pd.DataFrame(data, columns=custom_column_names)
bookData
```

Figure 2 Data from book table [OUT PUT]

[14]:	book_id	title	isbn13	language_id	num_pages	publication_date	publisher_id
0	1	The World's First Love: Mary Mother of God	8987059752	2	276	1996-09-01	1010
1	2	The Illuminati	20049130001	1	352	2004-10-04	1967
2	3	The Servant Leader	23755004321	1	128	2003-03-11	1967
3	4	What Life Was Like in the Jewel in the Crown: ...	34406054602	1	168	1999-09-01	1978
4	5	Cliffs Notes on Aristophanes' Lysistrata The ...	49086007763	1	80	1983-12-29	416

NOTE: to select data from other tables, we use the same python codes, after specifying the table name. **Transforming data to fit the warehouse schema.**

###check for null values

```
selectedData.isnull().sum()
```

###replace null values with Unknown

```
selectedData.fillna("Unknown", inplace=True)
```

###check data types

```
selectedData.dtypes()
```

###Data from book table by picking same columns

```
New_BookData = bookData[['book_id','language_id']]
```

```
New_BookData2 = bookData[['publisher_id','isbn13','publication_date','num_pages']]
```

Output for new variables [New_BookData and New_BookData2]

New_BookData.head(5)			New_BookData2.head(5)				
book_id	language_id		publisher_id	isbn13	publication_date	num_pages	
0	1	2	0	1010	8987059752	1996-09-01	276
1	2	1	1	1967	20049130001	2004-10-04	352
2	3	1	2	1967	23755004321	2003-03-11	128
3	4	1	3	1978	34406054602	1999-09-01	168
4	5	1	4	416	49086007763	1983-12-29	80

###Transform Data for DimBook table and rename columns names [Codes]

```
current_date = datetime.now()
```

```
New_BookData =
```

```
New_BookData.rename(columns= {'book_id':'book_id_BK','language_id':'language_id_BK'})
```

```
Newbook_authorData = Newbook_authorData.rename(columns= {'author_id':'author_id_BK'})
```

```
New_BookData2 = New_BookData2.rename(columns=
```

```
{ 'publisher_id':'publisher_id_BK','isbn13':'isbn13','publication_date':'publication_date','num_pages':  
'num_pages'})
```

```
DimBookData = pd.concat([New_BookData,
```

```
Newbook_authorData,
```

```
New_BookData2,
```

```

Newbook_languageData,
NewpublisherData,
NewauthorData
],
axis=1, join='inner')
DimBookData['source_system_code'] = 'gravity_books'
DimBookData['start_date'] = current_date
DimBookData['end_date'] = pd.to_datetime('2025-12-31')
DimBookData['is_current'] = 1

```

DimBookData

DimBookData.head(5)

book_id_BK	language_id_BK	author_id_BK	publisher_id_BK	isbn13	publication_date	num_pages	language_name	publisher_name	author_name	source_system_code
1	2	2778	1010	8987059752	1996-09-01	276	English	10/18	A. Bartlett Giamatti	gravity_boo
2	1	5049	1967	20049130001	2004-10-04	352	United States English	1st Book Library	A. Elizabeth Delany	gravity_boo
3	1	4902	1967	23755004321	2003-03-11	128	French	1st World Library	A. Merritt	gravity_boo
4	1	8651	1978	34406054602	1999-09-01	168	Spanish	A & C Black (Childrens books)	A. Roger Merrill	gravity_boo
5	1	8904	416	49086007763	1983-12-29	80	British English	A Harvest Book/Harcourt Inc.	A. Walton Litz	gravity_boo

###Transform Data for DimCustomer [Codes]

```

NewCustomerData_customer_id =
NewCustomerData_customer_id.rename(columns={'customer_id':'customer_id_BK'})
NewaddressData_address_id =
NewaddressData_address_id.rename(columns={'address_id':'address_id_BK'})
NewcountryData_country_id =
NewcountryData_country_id.rename(columns={'country_id':'country_id_BK'})
Newcustomer_address_address_id =
Newcustomer_address_address_id.rename(columns={'address_id':'status_id_BK'})
Newcustomer_address_status_id =
Newcustomer_address_status_id.rename(columns={'status_id':'address_status'})
DimCustomerData = pd.concat([NewCustomerData_customer_id,
NewaddressData_address_id,
NewcountryData_country_id,
Newcustomer_address_address_id,
countryData,
NewCustomerData,
NewaddressData,
Newcustomer_address_status_id
],
axis=1, join='inner')
DimCustomerData['source_system_code'] = 'gravity_books'
DimCustomerData['start_date'] = current_date
DimCustomerData['end_date'] = pd.to_datetime('2025-12-31')
DimCustomerData['is_current'] = 1

```

DimCustomerData

DimCustomerData											
[457]:	customer_id_BK	address_id_BK	country_id_BK	status_id_BK	country_id	country_name	first_name	last_name	email	street_name	city
0	1	1	1	57	1	Afghanistan	Ursola	Purdy	upurdy0@cdbaby.com	Glacier Hill Avenue	Torbat-e Jam
1	2	2	2	128	2	Netherlands Antilles	Ruthanne	Vatini	rvatini1@fema.gov	Dottie Junction	Beaumont
2	3	3	3	892	3	Albania	Reidar	Turbitt	rturbitt2@geocities.jp	Ramsey Avenue	Cayambe
3	4	4	4	365	4	Algeria	Rich	Kirszt	rkirszt3@jalbum.net	Thackeray Junction	Caldas
4	5	5	5	718	5	Andorra	Carline	Kupis	ckupis4@tamu.edu	2nd Park	Nqunquru

###Transform Data for DimDate [Codes]

```

current_date = datetime.now()
today = datetime.today().date()
# set DimDateData_is_current columns by comparing order_historyData['status_date'] value and to Day
order_historyData['is_current'] = (order_historyData['status_date'].dt.date == today).astype(int)
DimDateData_is_current = order_historyData['is_current']
# set DimDateData_end_date columns by adding 5 years to order_historyData['status_date'] value
order_historyData['end_date'] = order_historyData['status_date']
DimDateData_end_date = order_historyData['end_date'] + pd.DateOffset(years=5)
DimDateData = pd.concat([order_historyData_year,
                          order_historyData_month,
                          order_historyData_day
                          ],
                          axis=1, join='inner')
DimDateData['start_date'] = order_historyData['status_date']
DimDateData['end_date'] = DimDateData_end_date
DimDateData['is_current'] = DimDateData_is_current

```

DimDateData

DimDateData								
[468]:	year	month	day	start_date		end_date		is_current
0	2024	1	23	2024-01-23 06:09:56.030	2029-01-23 06:09:56.030			0
1	2022	8	16	2022-08-16 10:05:44.030	2027-08-16 10:05:44.030			0
2	2023	6	6	2023-06-06 23:43:42.030	2028-06-06 23:43:42.030			0
3	2024	8	1	2024-08-01 01:35:24.030	2029-08-01 01:35:24.030			0
4	2023	5	16	2023-05-16 07:28:18.030	2028-05-16 07:28:18.030			0

###Transform Data for DimOrder [Codes]

```

Newshipping_methodData_method_id_name =
Newshipping_methodData_method_id_name.rename(columns={'method_id': 'method_id_bk'})
cust_orderData_order_date =
cust_orderData_order_date.rename(columns={'order_date': 'start_date'})
cust_orderData_order_date =
cust_orderData_order_date.rename(columns={'order_date': 'start_date'})
today = datetime.today().date()

```

```

order_historyData['end_date'] = order_historyData['status_date']
DimOrderData_end_date = order_historyData['end_date'] + pd.DateOffset(years=3)
DimOrderData = pd.concat([ Newshipping_methodData_method_id_name,
                           cust_orderData_order_date
                           ],
                           axis=1, join='inner')
DimOrderData['source_system_code'] = 'gravity_books Database'
DimOrderData['start_date'] = order_historyData['status_date']
DimOrderData['end_date'] = DimOrderData_end_date
DimOrderData['is_current'] = DimDateData_is_current

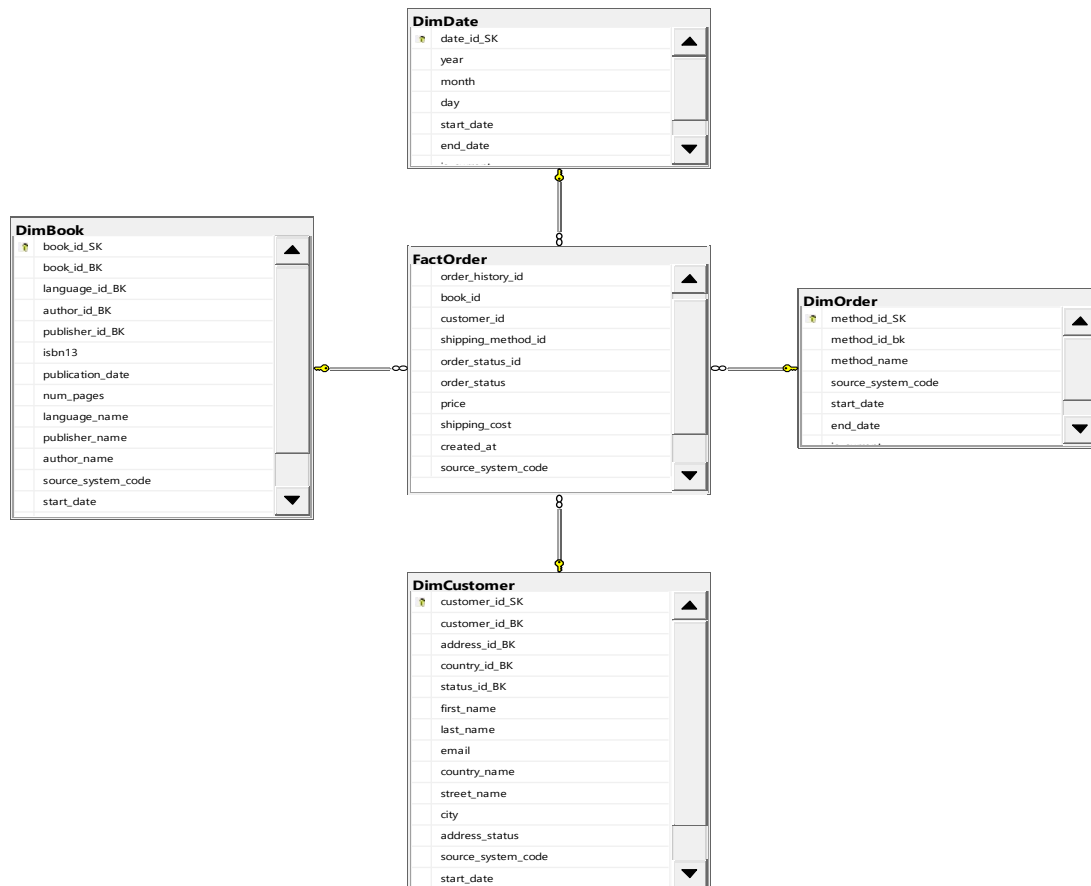
```

DimOrderData

	DimOrderData						
[463]:	method_id_bk	method_name	start_date	source_system_code	end_date	is_current	
0	1	Standard	2024-01-23 06:09:56.030	gravity_books Database	2027-01-23 06:09:56.030	0	
1	2	Priority	2022-08-16 10:05:44.030	gravity_books Database	2025-08-16 10:05:44.030	0	
2	3	Express	2023-06-06 23:43:42.030	gravity_books Database	2026-06-06 23:43:42.030	0	
3	4	International	2024-08-01 01:35:24.030	gravity_books Database	2027-08-01 01:35:24.030	0	

1. Loading data into the data warehouse.

Data warehouse schema



Database Connection for Data warehouse

```
conn = pyodbc.connect("
    DRIVER={ODBC Driver 17 for SQL Server};
    SERVER=DESKTOP-MF69IV4\\SQLEXPRESS;
    DATABASE= Data_warehouse;
    Trusted_Connection=yes
")
```

Load Data into DimCustomer Table

```
cursor = conn.cursor()
insert_query = """INSERT INTO DimCustomer (customer_id_BK, address_id_BK, country_id_BK, tatus_id_BK,
    first_name, last_name, email, country_name, street_name, city, address_status,
    source_system_code, start_date, end_date, is_current
    )
    VALUES
    (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)"""

for index, row in DimCustomerData.iterrows():
    cursor.execute(insert_query, (row['customer_id_BK'], row['address_id_BK'], row['country_id_BK'],
        row['status_id_BK'], row['first_name'], row['last_name'], row['email'], row['country_name'],
        row['street_name'], row['city'], row['address_status'], row['source_system_code'],
        row['start_date'], row['end_date'], row['is_current']
    ))
conn.commit()
print("Data inserted successfully into dbo.DimCustomer table.")
```

Retrieve Data of DimDate from Data warehouse

```
cursor.execute("SELECT * FROM DimDate")
rows = cursor.fetchall()
data = [list(row) for row in rows]
DimDate_column_names = [column[0] for column in cursor.description]
# Define your own column names
DimDate_column_names = ['date_id_SK', 'year', 'month', 'day', 'start_date', 'end_date', 'is_current']
# Create a Pandas DataFrame with your custom column names
DimDateData = pd.DataFrame(data, columns = DimDate_column_names )
```

DimDateData [OUTPUT]

DimDateData

	date_id_SK	year	month	day	start_date	end_date	is_current
0	1	2024	1	23	2024-01-23	2029-01-23	False
1	2	2022	8	16	2022-08-16	2027-08-16	False
2	3	2023	6	6	2023-06-06	2028-06-06	False
3	4	2024	8	1	2024-08-01	2029-08-01	False
4	5	2023	5	16	2023-05-16	2028-05-16	False

Foreign Keys From Dimention Tables

```
DimDateData_date_id_SK = DimDateData[['date_id_SK']]
DimBookData_book_id = DimBookData[['book_id_SK']]
DimCustomerData_customer_id_SK = DimCustomerData[['customer_id_SK']]
DimOrderData_method_id_SK = DimOrderData[['method_id_SK']]

### Transform Data for FactOrder [Codes]
current_date = datetime.now()
Neworder_lineData_line_id = Neworder_lineData_line_id.rename(columns={'order_line':'line_id'})
NewDimDateData_date_id_SK = DimDateData_date_id_SK.rename(columns={'date_id_SK':'order_history_id'})
NewDimBookData_book_id = DimBookData_book_id.rename(columns={'book_id_SK':'book_id'})
NewDimCustomerData_customer_id =
DimCustomerData_customer_id_SK.rename(columns={'customer_id_SK':'customer_id'})
NewDimOrderData_method_id_SK =
DimOrderData_method_id_SK.rename(columns={'method_id_SK':'shipping_method_id'})
Neworder_statusData_status_id = Neworder_statusData_status_id.rename(columns={'status_id':'order_status_id'})
Neworder_statusData_status_value =
Neworder_statusData_status_value.rename(columns={'status_value':'order_status'})
Neworder_lineData_price = Neworder_lineData_price.rename(columns={'price':'price'})
Newshipping_methodData_cost = Newshipping_methodData_cost.rename(columns={'cost':'shipping_cost'})
Newcust_orderData_created_at = cust_orderData_order_date.rename(columns={'start_date':'created_at'})
FactOrderData = pd.concat([
    Neworder_lineData_line_id,
    NewDimDateData_date_id_SK,
    NewDimBookData_book_id,
    NewDimCustomerData_customer_id,
    NewDimOrderData_method_id_SK,
    Neworder_statusData_status_id,
    Neworder_statusData_status_value,
    Neworder_lineData_price,
    Newshipping_methodData_cost,
    Newcust_orderData_created_at
],
    axis=1, join='inner')
FactOrderData['source_system_code'] = 'gravity_books Database'
FactOrderData
```

FactOrderData												
[1000...	line_id	order_history_id	book_id	customer_id	shipping_method_id	order_status_id	order_status	price	shipping_cost	created_at	source_system_code	
0	1	1	1	3395	1	1	Order Received	3.62	5.90	2024-01-23 04:45:34.030	gravity_books Database	
1	2	2	2	3396	2	2	Pending Delivery	2.29	8.90	2022-08-16 07:27:45.030	gravity_books Database	
2	3	3	3	3397	3	3	Delivery In Progress	11.50	11.90	2023-06-06 20:56:27.030	gravity_books Database	
3	4	4	4	3398	4	4	Delivered	9.60	24.50	2024-07-31 15:21:24.030	gravity_books Database	

Load Data into FactOrder Table

```
insert_query = """INSERT INTO FactOrder ( line_id, order_history_id, book_id, customer_id, shipping_method_id,
order_status_id, order_status, price, shipping_cost, created_at, source_system_code)
VALUES
(?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)"""

for index, row in FactOrderData.iterrows():
    cursor.execute(insert_query, ( row['line_id'], row['order_history_id'], row['book_id'], row['customer_id'],
    row['shipping_method_id'], row['order_status_id'], row['order_status'], row['price'], row['shipping_cost'],
    row['created_at'], row['source_system_code'],
    ))
conn.commit()
```

print("Data inserted successfully into dbo.FactOrder table.")

```
cursor.execute("SELECT * FROM FactOrder")
rows = cursor.fetchall()
data = [list(row) for row in rows]
DimFactOrder_column_names = [column[0] for column in cursor.description]
# Define your own column names
DimOrder_column_names = ['order_id_SK', 'line_id', 'order_history_id', 'book_id', 'customer_id', 'date_id_SK',
'order_status_id', 'order_status', 'price', 'shipping_cost', 'source_system_code', 'created_at']
# Create a Pandas DataFrame with your custom column names
FactOrderData = pd.DataFrame(data, columns = DimFactOrder_column_names )
```

FactOrderData

FactOrderData												
[1004]:	order_id_SK	line_id	order_history_id	book_id	customer_id	shipping_method_id	order_status_id	order_status	price	shipping_cost	created_at	source_system_code
0	1	1	1	1	3395	1	1	Order Received	3.62	5.90	2024-01-23 04:45:34.030	gravity_books Database
1	2	2	2	2	3396	2	2	Pending Delivery	2.29	8.90	2022-08-16 07:27:45.030	gravity_books Database
2	3	3	3	3	3397	3	3	Delivery In Progress	11.50	11.90	2023-06-06 20:56:27.030	gravity_books Database
3	4	4	4	4	3398	4	4	Delivered	9.60	24.50	2024-07-31 15:21:24.030	gravity_books Database